

Applied Machine Learning Systems - Report

AMLS SoSe26

Adrian Kraft - 498322

1. Task 1 - Data Preparation

The first task required writing code to analyze the given data and make certain transformation decisions based on that analysis. First, it was interesting to see how the data was distributed. The training data originally consisted of 6 classes: Class 0 consisted of the real images, and Classes 1 through 5 consisted of the AI-generated images, with each class representing a different AI model.

Each class was roughly the same size. However, since classes 1 through 5 were to be merged into a single class later on, the label distribution after merging was as follows: Label 0 accounted for about 1/6, and Label 1 for about 5/6. Overall, it was a very clean data set. There were hardly any duplicates, no null values, and no corrupted images that couldn't be read.

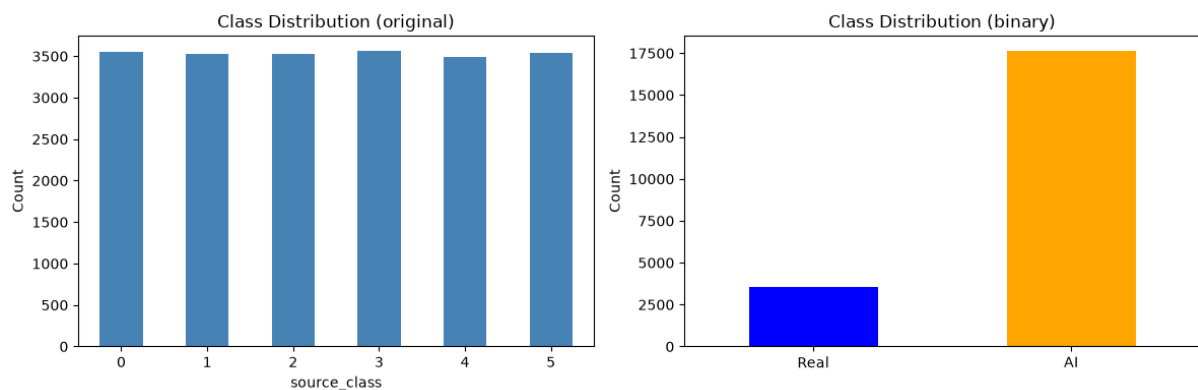


Figure 1: Class distribution before and after merging the labels

The most important step after that was to resize all the images to a uniform format. Most of the images were in 320×320px format (mostly from the AI Images class) and had 3 color channels. However, 320×320px is unnecessarily large and contains a lot of superfluous information, especially at the edges. This would have significantly slowed down processing during training without offering any expectation of higher accuracy, given the limited computation time. In fact, it would likely have had the opposite effect: since fewer images, or epochs would have been included in the training within the given 30-minute time limit, a poorer performance would have been expected. Therefore, I decided to crop each image to 128×128px.

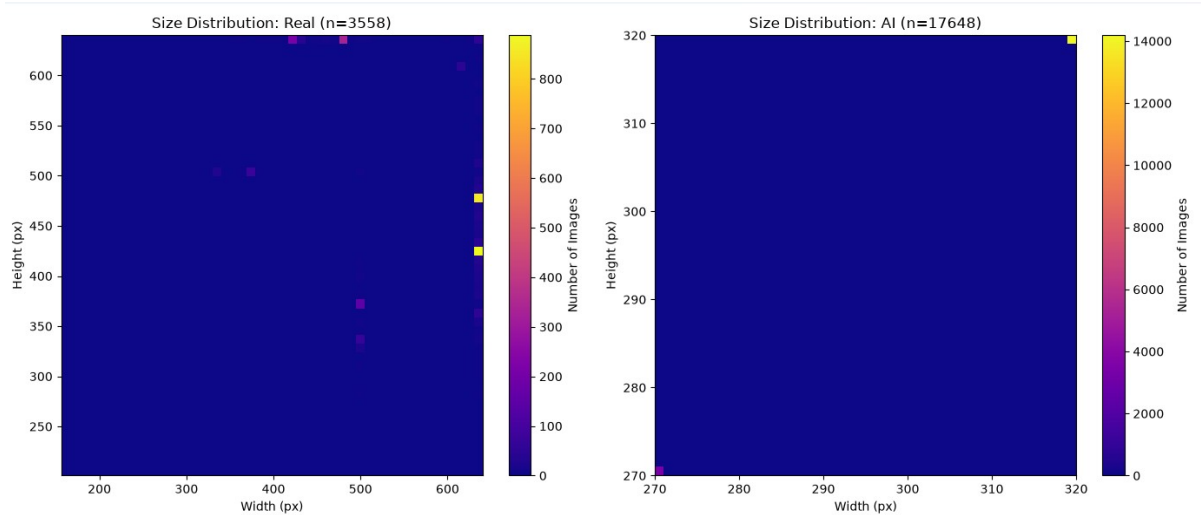


Figure 2: Width and height distribution of the images before cropping

This also raised the question of which 128×128 px window to crop from the image, or whether to use a cropping algorithm based on average pixel values. I ultimately opted for the simplest solution and chose the central 128×128 px section of each image. An alternative would have been to implement a kind of object recognition based on contrast that attempts to filter out the main subject. However, there were also reasons against this: first, this would require more development effort; second, statistically speaking, the main subject is already in the center of most images anyway; and third, the simple center section itself could contain a useful feature for example, that real images, are less likely to have a centered main subject than AI-generated images, which could be detected based on the contrasts within that window.

I also removed images that had a particularly extreme width to height ratio, since these likely would not have made for good training data when resized. In addition, I removed very small images (64×64 px or smaller), as they contained too little information for training. A total of 11 images were removed.

When analyzing the data, it also became apparent that AI-generated images contained significantly fewer outliers in terms of width and height. For example, almost all AI-generated images had a size of 320×320 px, making them among the largest images in the dataset. The width/height ratios of the real images were much more variable and were also less frequently squared. As a result, I added the original image widths and heights (before resizing to 128×128 px) as metadata to the dataframe. You could argue that this is an overfitting feature that helps provide a more precise result specifically in this dataset. However, you could also argue that AI-generated images are produced in a specific format far more frequently in reality than camera images, and that this feature would therefore occur in a similar way outside of this dataset as well, which is more likely in my perspective.

Additional Meta-Features (addendum)

Through further experimentation and research, I discovered additional metadata that can be extracted from the original images before JPEG compression and used as additional training features. The rationale behind this follows the same logic as with the baseline model (see there): these features capture image properties that are not readily accessible from the raw pixel values alone for a CNN with limited input resolution (128×128 px).

Specifically, four features were added. Noise Residual (mean and standard deviation) is the difference between the original image and a Gaussian-blurred version, capturing fine texture and noise patterns that may exhibit statistically different properties in AI-generated images compared to natural photographs. Edge density is the average edge strength of the image, calculated using an edge filter; this supplements the local contrast feature already used in the baseline model with a more global metric. Saturation is the average deviation of the RGB channels from their respective grayscale values; as observed in the baseline model, AI-generated images tend to have slightly higher saturation.

These four features were concatenated with the learned image features as an additional input vector just before the classifier, analogous to the image dimensions (width, height) already used. The integration of these additional features led to an improvement in the Recall_AI@FPR≤20% metric on the validation set of approximately 1 percentage point.

2. Task 2 - Modeling and Tuning under Time Constraints

For this task, you should first create a prepare script to reduce overhead during training and implement at least two different model families. I decided to use a baseline_model and a CNN model. To develop the baseline model, I first considered parameters that could reveal differences between the two labels. Ultimately, I chose global color distribution and contrast, local contrast, FFT analysis, original size, and saturation.

During the preparation phase, I decided to convert the cleaned Parquet files into uint8 tensors. The metadata is also included in the tensors and saved under 'artifacts/train_tensors1.pt', 'artifacts/validation_tensors1.pt', and 'artifacts/calibration_tensors1.pt'.

2.1 Baseline Model

Global color distribution and contrast: Here, I simply calculated the mean and standard deviation for each pixel's color value, for both labels. However, analysis of these values revealed that the two distributions were nearly identical, so these were generally rather weak features.

Local contrasts: Here, I divided each image into 8x8-pixel grids and calculated the standard deviation from the average RGB value for each grid. Then I calculated the average standard deviation across all grids in an image and then again the average across all images in a class. This was a stronger feature.

FFT Analysis: The goal here was to detect FFT noise in the images and thus be able to assess it. This is a distinction that is often cited as a feature due to the generation of AI images (see: <https://emerginginvestigators.org/articles/24-270/pdf>). However, this also turned out to be a rather weak feature.

Original Size: As already mentioned in Task 1, there were notable differences in the distribution of image formats between the two classes. AI images had more uniform formats and tended to be more square. The real images, on the other hand, had greater variance in height and width and also included more high-resolution images.

Saturation: Here, we looked at how much the RGB values deviate from gray values. A gray value is a value where the red, blue, and green values are equal. If there is a greater variance from the mean, the saturation is also higher. The assumption here was that AI-generated images tend to produce images

with higher saturation. This turned out to be true. However, the difference here wasn't enormous either, so it was also a rather weak feature.

2.2 Baseline Results

After training the feature weights, the model initially achieved an accuracy of 83%. At first, I thought I had apparently selected very good features. Then, however, I realized (and was disillusioned) that because the training and validation data were unbalanced, the model simply always predicted “AI image”, thereby achieving a score of 5/6, or 83%.

After I balanced the datasets, which also involved removing a large portion of the data, the model achieved an accuracy of around 67% with an FPR of 0.324. This meant the model did not sufficiently meet the requirements, so we had to move on to a different family, the CNN models.

2.3 CNN Architecture

I used the reference architecture proposed in Appendix B as a starting point, which I expanded in several ways:

Batch normalization: A BatchNorm2d layer was added after each convolutional layer. This normalizes the activations within a batch and stabilizes training, which was particularly relevant given the training instabilities described in the next section (significant fluctuations in validation accuracy between epochs).

Dual Pooling: Instead of using only AdaptiveAvgPool2d (average across all spatial positions), I also added AdaptiveMaxPool2d (maximum value) and concatenated both results. The basic idea: average pooling captures how strongly a pattern is represented on average across the entire image, while max pooling captures whether a pattern occurs particularly strongly at at least one location in the image, such as local compression or generation artifacts that do not affect the entire image. This doubles the number of features before the classifier (from 4k to 8k).

Enhanced Classifier: Instead of a single linear layer, I added an additional hidden layer (128 neurons) with ReLU activation, as well as two dropout layers ($p=0.5$). This increases the model's capacity compared to the baseline, while the dropout layers simultaneously act as regularization against overfitting—an aspect that is completely missing from the baseline architecture.

Metadata Integration: As described in Task 1, the analysis of the original image dimensions revealed a striking difference between the classes. To make this signal directly usable, the normalized original widths and heights are concatenated with the image features as an additional 2-dimensional vector shortly before the final classifier. The model thus combines learned visual features with an explicit, tabular feature.

2.4 Training Configuration

AdamW was used for training with a learning rate of $5e-4$ and `weight_decay=0.02`. The latter penalizes large weights in addition to dropout regularization. The learning rate was deliberately set lower than the originally tested value of $1e-3$, as higher learning rates sometimes resulted in sharp drops in validation accuracy between individual epochs, indicating training instability. Additionally, gradient clipping (`max_norm=1.0`) was implemented to prevent individual gradient updates that are too large, which could contribute to this instability.

A ReduceLROnPlateau scheduler automatically reduces the learning rate as soon as the target metric (Recall_AI@FPR<=20%) fails to improve over several epochs.

Since the model in train.py is trained using a fixed time budget (rather than a fixed number of epochs), the best model to date (based on Recall_AI@FPR<=20% on the validation dataset) is saved after each epoch; in addition, when 95% of the time budget is reached, the latest intermediate state is saved separately to ensure a usable checkpoint even in the event of an abrupt termination.

FocalLoss

Instead of the standard cross-entropy loss, a focal loss was used ($\gamma = 1.0$). This loss function weights the error contribution of individual examples based on the model's confidence: images that are already classified correctly with a high degree of certainty contribute less and less to the total error, while more difficult, uncertain cases remain more heavily weighted. As a result, the training focus automatically shifts over the course of the epochs from patterns that have already been learned well to the remaining borderline cases.

Pruning

In addition, after epoch 10, the model's confidence (pt) for the correct class is measured once for each training image. The 10% of images with the highest confidence—that is, those that the model already classifies correctly and reliably—are then permanently removed from the training dataset. Conceptually, this approach is similar to Focal Loss, but here it is primarily aimed at shortening the epoch duration: Thanks to the smaller dataset, 1–2 additional epochs could be computed within the remaining training time. However, no distinct, significant learning effect beyond the time savings resulting from the smaller dataset was clearly observable.

2.5 Results

One of the biggest problems when creating the model with train.py was the limited runtime. At first, I was loading the .parquet files during training, but this was too slow, and I was only able to complete 8–9 epochs. Then I switched to tensors. Initially, I used float32 tensors, but these temporarily consumed up to 8 GB of RAM, which was not permitted by the task specifications. Therefore, I ultimately switched to uint8 tensors. This made training significantly faster, since I could create the tensors during the prepare runtime.

I also had issues with the data weighting. At first, I used a parameter that assigned a loss factor of 5 when an AI image was misclassified, while misclassified real images were still penalized with a loss factor of 1. However, this didn't work well because it likely made the training too unstable. So I switched to balancing the datasets; while this meant I no longer considered two-thirds of my data, it consistently yielded better results. Then it was mostly about hyperparameter tuning, during which I experimented extensively with the learning rate and batch size. Here, too, I achieved the best results with a batch size of 48 and a learning rate of $5e-4$. After I found the threshold using the calibration set, I arrived at a Recall_AI@FPR<=20% value of 0.7895.

```
load cached tensors
Training loaded: 21043 images
Validation loaded: 1124 images
Balanced: 7034 images (3517 Real, 3517 AI)
Epoch 1/50 - Train Loss: 0.3761 - Val Loss: 0.3123 - Val Acc: 0.7456 - Recall_AI@FPR<=20%: 0.3686 - LR: 0.000500 - Time: 87s
!!! new best Model (Recall_AI@FPR<=20%: 0.3686)
Epoch 2/50 - Train Loss: 0.3333 - Val Loss: 0.3036 - Val Acc: 0.7473 - Recall_AI@FPR<=20%: 0.4348 - LR: 0.000500 - Time: 171s
!!! new best Model (Recall_AI@FPR<=20%: 0.4348)
Epoch 3/50 - Train Loss: 0.3158 - Val Loss: 0.2857 - Val Acc: 0.7527 - Recall_AI@FPR<=20%: 0.5769 - LR: 0.000500 - Time: 258s
!!! new best Model (Recall_AI@FPR<=20%: 0.5769)
Epoch 4/50 - Train Loss: 0.2964 - Val Loss: 0.2737 - Val Acc: 0.7322 - Recall_AI@FPR<=20%: 0.6261 - LR: 0.000500 - Time: 341s
!!! new best Model (Recall_AI@FPR<=20%: 0.6261)
Epoch 5/50 - Train Loss: 0.2860 - Val Loss: 0.4849 - Val Acc: 0.3870 - Recall_AI@FPR<=20%: 0.5491 - LR: 0.000500 - Time: 430s
Epoch 6/50 - Train Loss: 0.2815 - Val Loss: 0.2015 - Val Acc: 0.8319 - Recall_AI@FPR<=20%: 0.6528 - LR: 0.000500 - Time: 512s
!!! new best Model (Recall_AI@FPR<=20%: 0.6528)
Epoch 7/50 - Train Loss: 0.2701 - Val Loss: 0.2598 - Val Acc: 0.7429 - Recall_AI@FPR<=20%: 0.6560 - LR: 0.000500 - Time: 595s
!!! new best Model (Recall_AI@FPR<=20%: 0.6560)
Epoch 8/50 - Train Loss: 0.2675 - Val Loss: 0.2204 - Val Acc: 0.8123 - Recall_AI@FPR<=20%: 0.6432 - LR: 0.000500 - Time: 680s
Epoch 9/50 - Train Loss: 0.2624 - Val Loss: 0.3096 - Val Acc: 0.6593 - Recall_AI@FPR<=20%: 0.6645 - LR: 0.000500 - Time: 759s
!!! new best Model (Recall_AI@FPR<=20%: 0.6645)
Epoch 10/50 - Train Loss: 0.2531 - Val Loss: 0.2056 - Val Acc: 0.8265 - Recall_AI@FPR<=20%: 0.6549 - LR: 0.000500 - Time: 838s
Pruning: 6330 of 7034 images kept (Threshold: 0.854)
Epoch 11/50 - Train Loss: 0.2506 - Val Loss: 0.1940 - Val Acc: 0.8470 - Recall_AI@FPR<=20%: 0.7179 - LR: 0.000500 - Time: 921s
!!! new best Model (Recall_AI@FPR<=20%: 0.7179)
Epoch 12/50 - Train Loss: 0.2668 - Val Loss: 0.3593 - Val Acc: 0.5667 - Recall_AI@FPR<=20%: 0.6859 - LR: 0.000500 - Time: 990s
Epoch 13/50 - Train Loss: 0.2601 - Val Loss: 0.2643 - Val Acc: 0.7420 - Recall_AI@FPR<=20%: 0.7051 - LR: 0.000500 - Time: 1064s
Epoch 14/50 - Train Loss: 0.2599 - Val Loss: 0.2589 - Val Acc: 0.7358 - Recall_AI@FPR<=20%: 0.7019 - LR: 0.000500 - Time: 1140s
Epoch 15/50 - Train Loss: 0.2566 - Val Loss: 0.3240 - Val Acc: 0.6192 - Recall_AI@FPR<=20%: 0.7158 - LR: 0.000250 - Time: 1216s
Epoch 16/50 - Train Loss: 0.2450 - Val Loss: 0.2189 - Val Acc: 0.8114 - Recall_AI@FPR<=20%: 0.7489 - LR: 0.000250 - Time: 1290s
!!! new best Model (Recall_AI@FPR<=20%: 0.7489)
Epoch 17/50 - Train Loss: 0.2451 - Val Loss: 0.2014 - Val Acc: 0.8301 - Recall_AI@FPR<=20%: 0.7457 - LR: 0.000250 - Time: 1365s
Epoch 18/50 - Train Loss: 0.2447 - Val Loss: 0.1991 - Val Acc: 0.8434 - Recall_AI@FPR<=20%: 0.7671 - LR: 0.000250 - Time: 1495s
!!! new best Model (Recall_AI@FPR<=20%: 0.7671)
Epoch 19/50 - Train Loss: 0.2388 - Val Loss: 0.2253 - Val Acc: 0.7963 - Recall_AI@FPR<=20%: 0.7628 - LR: 0.000250 - Time: 1566s
Epoch 20/50 - Train Loss: 0.2379 - Val Loss: 0.1936 - Val Acc: 0.8372 - Recall_AI@FPR<=20%: 0.7489 - LR: 0.000250 - Time: 1642s
Epoch 21/50 - Train Loss: 0.2366 - Val Loss: 0.2643 - Val Acc: 0.7367 - Recall_AI@FPR<=20%: 0.7895 - LR: 0.000250 - Time: 1716s
!!! new best Model (Recall_AI@FPR<=20%: 0.7895)
Timeout reached
Training finished
```

Figure 3: Training and validation runs over the epochs

calibrated threshold (on calibration-set): 0.47 (Recall_AI: 0.777)

validation with Threshold 0.47:

FPR (Validation): 0.176

Recall_AI (Validation): 0.771

	precision	recall	f1-score	support
0	0.42	0.82	0.56	188
1	0.96	0.77	0.85	936
accuracy			0.78	1124
macro avg	0.69	0.80	0.71	1124
weighted avg	0.87	0.78	0.80	1124

--- Threshold-Sweep on Validation

Threshold 0.30: FPR=0.707, Recall_AI=0.964

Threshold 0.31: FPR=0.665, Recall_AI=0.958

Threshold 0.32: FPR=0.649, Recall_AI=0.953

Threshold 0.33: FPR=0.596, Recall_AI=0.950

Threshold 0.34: FPR=0.574, Recall_AI=0.947

Threshold 0.35: FPR=0.543, Recall_AI=0.939

Threshold 0.36: FPR=0.516, Recall_AI=0.934

Threshold 0.37: FPR=0.473, Recall_AI=0.922

Threshold 0.38: FPR=0.436, Recall_AI=0.911

Threshold 0.39: FPR=0.415, Recall_AI=0.895

Threshold 0.40: FPR=0.388, Recall_AI=0.881

Threshold 0.41: FPR=0.372, Recall_AI=0.869

Threshold 0.42: FPR=0.319, Recall_AI=0.860

Threshold 0.43: FPR=0.293, Recall_AI=0.841

Threshold 0.44: FPR=0.261, Recall_AI=0.823

Threshold 0.45: FPR=0.229, Recall_AI=0.803

Threshold 0.46: FPR=0.197, Recall_AI=0.790

Threshold 0.47: FPR=0.176, Recall_AI=0.771

Threshold 0.48: FPR=0.176, Recall_AI=0.743

Threshold 0.49: FPR=0.160, Recall_AI=0.728

Threshold 0.50: FPR=0.138, Recall_AI=0.712

Threshold 0.51: FPR=0.133, Recall_AI=0.690

Threshold 0.52: FPR=0.117, Recall_AI=0.668

Threshold 0.53: FPR=0.101, Recall_AI=0.637

Threshold 0.54: FPR=0.085, Recall_AI=0.608

Threshold 0.55: FPR=0.069, Recall_AI=0.589

Threshold 0.56: FPR=0.064, Recall_AI=0.568

Threshold 0.57: FPR=0.059, Recall_AI=0.545

Threshold 0.58: FPR=0.053, Recall_AI=0.531

Threshold 0.59: FPR=0.053, Recall_AI=0.506

Figure 4: Threshold Sweep: FPR and Recall_AI as a Function of Threshold

2.6 Error Analysis

A Recall_AI@FPR $\leq$ 20% of approximately 0.77 was achieved (with calibrated threshold). This is slightly below the target of 0.8. This can be explained by weaknesses in the data preprocessing or model architecture that contribute to the actual instability. To clarify this specifically rather than merely speculating, the next step would be to use explainability methods (e.g., Grad-CAM on the conv-feature maps) to examine what the model actually bases its decisions on, for example, whether it learns robust, substantive patterns or relies on unstable, slightly noisy signals. This would allow us to distinguish between a mere evaluation artifact and a genuine model or data problem, rather than conflating the two. Another idea would be to do more hyperparameter fine-tuning to close the remaining gap. I also did not analyze how the model would perform with more data and computation time, but these factors should be among the main drivers of significantly better performance. In addition, the AI images that were not used in training were selected at random. We could experiment with different random seeds or a specific function to determine the most suitable images to see if the model would achieve better performance, but the increase in precision would likely not be very significant.

3. Task 3 - Robustness to Image Degradation

The goal of this task was to make the Task 2 model more robust against image degradation, such as that which can occur in practice due to compression, scaling, or blurring. Instead of training from scratch, training was continued from the Task-2 checkpoint (fine-tuning) to reuse the representations already learned and save computation time within the time budget. To achieve this, the learning rate was deliberately set lower ($2e-4$ instead of $5e-4$), and the same class-weighted loss as in the final Task-2 version was retained.

As an augmentation strategy, during training, one of four randomly selected degradations is applied to each image with a 50% probability, in addition to a horizontal flip: JPEG recompression (random quality 30–90), Gaussian blur (standard deviation between 0.5–2.0), downscaling/upscaling (scaling factor 0.4–0.8), and mild Gaussian noise. These four variants directly cover the distortion factors specified in the task description (scaling, compression, blurring). Since the decision on whether and

which degradation to apply is made anew and randomly at each epoch, the model is exposed to many different degradation variants over time, rather than becoming accustomed to a fixed damaged version.

Unlike in Task 2, the best model is selected during training based on Recall_AI@FPR≤20% on data/validation_augmented/ rather than on the clean validation set, since the very purpose of this task is to assess robustness against data degradation. This may result in a slight trade-off compared to the pure clean performance of the Task 2 model, which is demonstrated in the results section with specific figures for both sets.

For the final calibration, as previously identified and corrected as a methodological error in Task 2, the threshold was determined exclusively on a separate calibration set (here: a combination of data/calibration/ and data/calibration_augmented/) and applied unchanged to both validation sets to obtain an unbiased estimate.

```
Balanced: 7034 images (3517 Real, 3517 AI)
Task-2-Checkpoint loaded, start fine-tuning with robustness augmentation
Epoch 1/50 - Train Loss: 0.2448 - Recall_clean: 0.7532 - Recall_augmented: 0.5848 - LR: 0.000100 - Time: 79s
!!! new best model (Recall_augmented: 0.5848)
Epoch 2/50 - Train Loss: 0.2352 - Recall_clean: 0.7650 - Recall_augmented: 0.5806 - LR: 0.000100 - Time: 160s
Epoch 3/50 - Train Loss: 0.2328 - Recall_clean: 0.7489 - Recall_augmented: 0.5464 - LR: 0.000100 - Time: 246s
Epoch 4/50 - Train Loss: 0.2337 - Recall_clean: 0.7511 - Recall_augmented: 0.5806 - LR: 0.000100 - Time: 337s
Epoch 5/50 - Train Loss: 0.2331 - Recall_clean: 0.7874 - Recall_augmented: 0.5646 - LR: 0.000050 - Time: 429s
Epoch 6/50 - Train Loss: 0.2273 - Recall_clean: 0.7650 - Recall_augmented: 0.5752 - LR: 0.000050 - Time: 534s
Epoch 7/50 - Train Loss: 0.2284 - Recall_clean: 0.7895 - Recall_augmented: 0.5635 - LR: 0.000050 - Time: 626s
Epoch 8/50 - Train Loss: 0.2199 - Recall_clean: 0.7724 - Recall_augmented: 0.5859 - LR: 0.000050 - Time: 717s
!!! new best model (Recall_augmented: 0.5859)
Epoch 9/50 - Train Loss: 0.2232 - Recall_clean: 0.7767 - Recall_augmented: 0.5464 - LR: 0.000050 - Time: 810s
Epoch 10/50 - Train Loss: 0.2227 - Recall_clean: 0.7938 - Recall_augmented: 0.5891 - LR: 0.000050 - Time: 904s
!!! new best model (Recall_augmented: 0.5891)
Epoch 11/50 - Train Loss: 0.2246 - Recall_clean: 0.7628 - Recall_augmented: 0.5624 - LR: 0.000050 - Time: 986s
Epoch 12/50 - Train Loss: 0.2188 - Recall_clean: 0.8045 - Recall_augmented: 0.5592 - LR: 0.000050 - Time: 1074s
Epoch 13/50 - Train Loss: 0.2253 - Recall_clean: 0.7724 - Recall_augmented: 0.5784 - LR: 0.000050 - Time: 1162s
Epoch 14/50 - Train Loss: 0.2221 - Recall_clean: 0.7778 - Recall_augmented: 0.5656 - LR: 0.000025 - Time: 1249s
Epoch 15/50 - Train Loss: 0.2245 - Recall_clean: 0.7585 - Recall_augmented: 0.5614 - LR: 0.000025 - Time: 1341s
Epoch 16/50 - Train Loss: 0.2230 - Recall_clean: 0.7692 - Recall_augmented: 0.6073 - LR: 0.000025 - Time: 1426s
!!! new best model (Recall_augmented: 0.6073)
Epoch 17/50 - Train Loss: 0.2178 - Recall_clean: 0.7949 - Recall_augmented: 0.5838 - LR: 0.000025 - Time: 1511s
Epoch 18/50 - Train Loss: 0.2194 - Recall_clean: 0.7746 - Recall_augmented: 0.5891 - LR: 0.000025 - Time: 1600s
Epoch 19/50 - Train Loss: 0.2179 - Recall_clean: 0.7810 - Recall_augmented: 0.5518 - LR: 0.000025 - Time: 1686s
Epoch 20/50 - Train Loss: 0.2137 - Recall_clean: 0.7788 - Recall_augmented: 0.5848 - LR: 0.000013 - Time: 1769s
Timeout reached
Training finished
```

Figure 5: Comparison of Recall_AI@FPR≤20% values on validation vs. validation_augmented

Results:

Unfortunately, the augmentation strategy did not lead to improved robustness on the augmented validation set, the model did not perform significantly better than the Task 2 CNN Model on degraded images.

Interestingly, the clean validation performance improved to approximately 0.80 Recall_AI@FPR≤20% during Task 3 fine-tuning, which suggests that the Task 2 model had not yet fully converged within the 30-minute training budget. Given more computation time, the Task 2 model itself would likely have reached this performance level as well.

The error analysis for Task 3 is largely similar to Task 2: the model still struggles with the fundamental trade-off between FPR and Recall_AI, and the training instability observed in Task 2 (fluctuating validation accuracy between epochs) persisted in Task 3 as well. The limited time budget remains the primary bottleneck, with more epochs, both robustness and clean performance would be expected to improve further.